Problemas de Decisão

Introdução

- Problemas fáceis vs problemas difíceis
  - Complexidade polinomial vs complexidade exponencial
- Objetivo: identificar problemas difíceis
  - Opção por heurísticas

Introdução

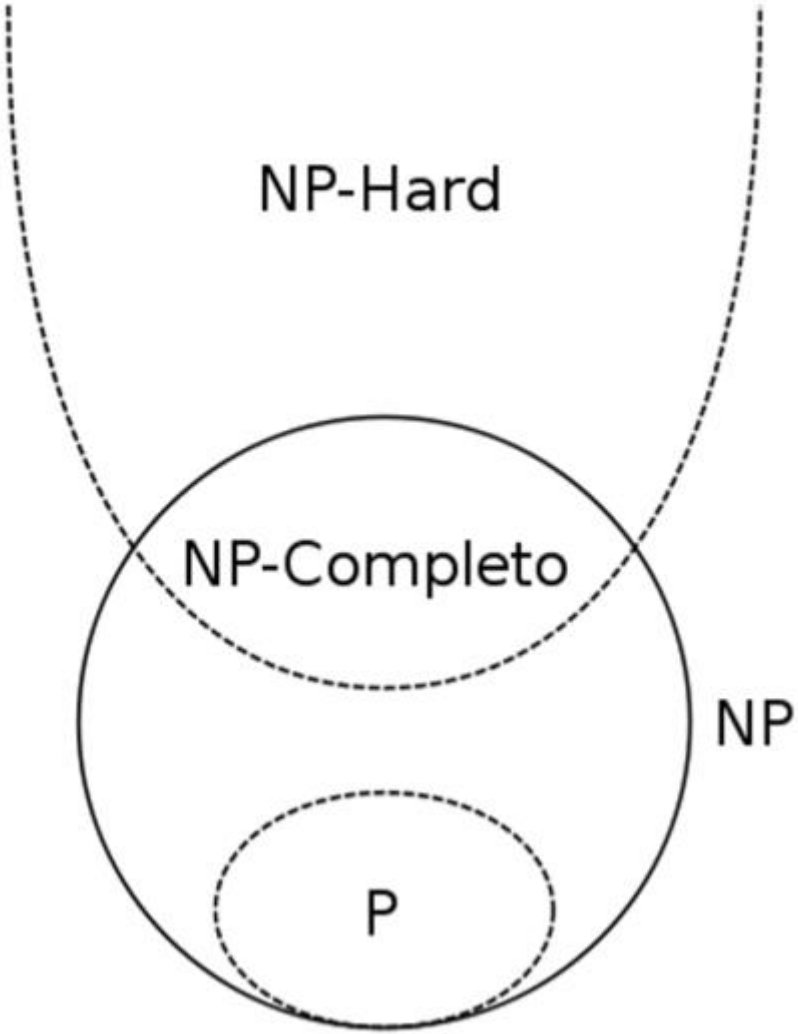
- Tratáveis (“fáceis”): podem ser resolvidos por um algoritmo de tempo polinomial;
- Intratáveis (“difíceis”): só podem ser resolvidos por algoritmos superpolinomiais (exponenciais, fatoriais, etc.)
- A complexidade de tempo da maioria dos problemas é polinomial ou exponencial.

Introdução

- Classe NP-Completo: problemas difíceis?
  - Em aberto, mas assumimos que sim

- Propriedade: um problema da classe NP poderá ser resolvido em tempo polinomial se e somente se todos os outros problemas em N P também puderem

Introdução



Problemas fáceis vs problemas difíceis

- As vezes a diferença é sútil:

- Dado um grafo G(V, E) e um inteiro k, existe uma coloração de G com no máximo k cores?

Problemas fáceis vs problemas difíceis

- As vezes a diferença é sútil:

- Dado um grafo G(V, E) e um inteiro k, existe uma coloração de G com no máximo k cores?
  - a) $k = 2$
    - Fácil: grafo bipartido
  - b) $k > 2$
    - Difícil: problema da coloração de vértices

Problemas fáceis vs problemas difíceis

- As vezes a diferença é sútil:

- Dado um grafo G(V, E) existe um ciclo simples que passa por todos os vértices sem repetir nenhum?

Problemas fáceis vs problemas difíceis

- As vezes a diferença é sútil:

- Dado um grafo G(V, E) existe um ciclo simples que passa por todos os vértices sem repetir nenhum?
  - a) grau máximo igual a 2
    - Fácil
  - b) grau máximo > 2
    - Difícil: problema do ciclo hamiltoniano

Problemas fáceis vs problemas difíceis

- As vezes a diferença é sútil:

- Dado um grafo G(V, E) e um inteiro k, existe uma cobertura de arestas de tamanho no máximo k?

- Dado um grafo G(V, E) e um inteiro k, existe uma cobertura de vértices de tamanho no máximo k?

Problemas fáceis vs problemas difíceis

- As vezes a diferença é sútil:

- Dado um grafo G(V, E) e um inteiro k, existe uma cobertura de arestas de tamanho no máximo k?
  - Fácil

- Dado um grafo G(V, E) e um inteiro k, existe uma cobertura de vértices de tamanho no máximo k?
  - Difícil

Classe NP

- Problemas de decisão (saída: sem ou não)
- Problemas de otimização vs problema de decisão:
  - Determinar o circuito simples que passa por todos os vértices com custo total mínimo (otimização)
  - Existe um circuito simples que passa por todos os vértices com custo menor ou igual a k (decisão)

Classe NP

- NP (Non-Deterministic Polynomial time): problemas de decisão cuja solução pode ser **verificada** em tempo polinomial com **algoritmo determinista**.

- Ou, problemas de decisão cuja solução pode ser **determinada** em tempo polinomial com **algoritmo não determinista**

Algoritmo não determinista

- Escolhe a alternativa que leva a solução ótima, caso uma exista
- Caso não exista, retorna qualquer resposta
- Complexidade: C(1)

Arcabouço teórico

Algoritmo não determinista

- Exemplo: pesquisar um elemento x em um vetor A de n elementos
  - Função que retorna um valor qualquer entre 1 e n e compara com x para verificar se houve sucesso ou não.
  - Algoritmo não determinista: C(1)
  - Pesquisa sequencial é C(n/2), deterministicamente

Algoritmo não determinista

- Exemplo: árvore geradora
  - Entrada: grafo conexo não direcionado
  - Algoritmo: **enquanto** uma aresta pode ser removida de forma que o grafo ainda se mantenha conectado,
  - Saída: uma árvore geradora (observe que um grafo pode possuir várias árvores geradoras e portanto há várias soluções para o problema)
- Obs.: não confundir com **árvore geradora mínima**.

Classes P, NP e NPC

- Classe P: Está na classe P se puder ser resolvido em tempo polinomial.
- Classe NP: Está na classe NP se puder ser verificado em tempo polinomial.
- Classe NP-Completo: Um problema X está na classe NP-Completo se:
  - X pertence à NP
  - Qualquer problema da classe NPC puder ser resolvido em tempo polinomial se, e somente se, X puder ser resolvido em tempo polinomial.

Problemas NP-Completos

- Ainda não foi descoberto um algoritmo polinomial para qualquer problema NP-Completo
- Ninguém até hoje foi capaz de demonstrar formalmente que os problemas NP-Completos são intratáveis
- Muitos problemas NP-Completos são muito parecidos com problemas polinomiais conhecidos

Transformação polinomial

- Informalmente, um problema está em NP-Completo se ele está em NP e é “tão difícil” quanto qualquer outro problema em NP

- Ser “tão difícil quanto”: transformação polinomial

Transformação polinomial

- Sejam A e B problemas de decisão
- Sejam α uma instância de A e β uma instância de B
- Suponha que B possa ser resolvido em tempo polinomial
- Suponha um algoritmo P que transforme α em β , com as seguintes propriedades:
  - a transformação é feita em tempo polinomial
  - as respostas são as mesmas, isto é, a resposta de β é SIM se, e somente se, a resposta para α for SIM

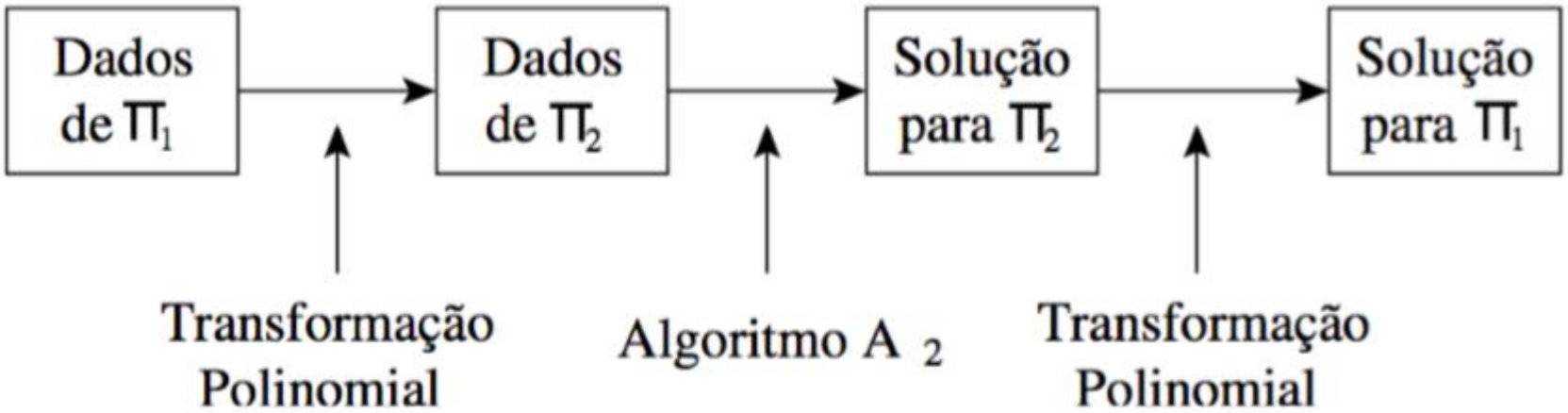
Transformação polinomial

- Um algoritmo polinomial para resolver α consiste em:
  - aplique P para transformar α em β
  - resolva β
  - use a resposta de β como resposta para α

Transformação polinomial

- Suponha que existam dois problemas de “sim/não” Π1 e Π2
- Suponha que um algoritmo A2 resolva Pi2
- Se for possível transformar Pi1 em Pi2 e a solução de Pi2 em solução de Pi1, então A2 pode ser utilizado para resolver Pi1.
- Se pudermos realizar as transformações nos dois sentidos em tempo polinomial, então Pi1 é polinomialmente transformável em Pi2.

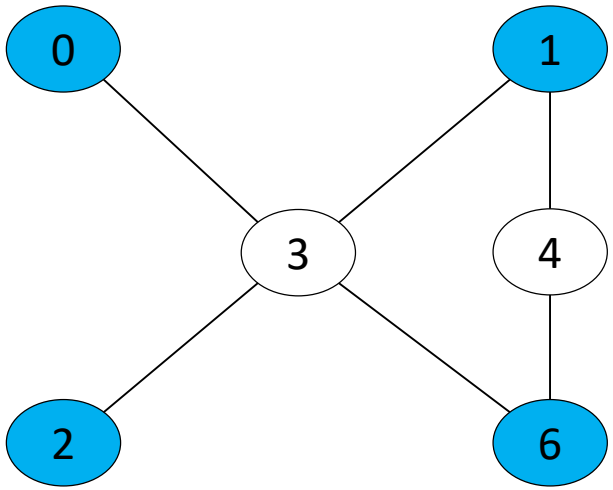
Transformação polinomial



- Esse conceito é importante para definir a classe NP-completo

Conjuntos Independentes

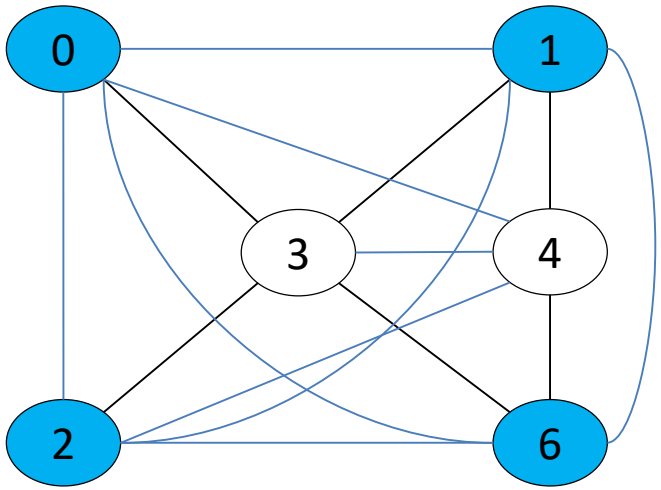
- Exemplo de cardinalidade 4:
  - $S = \{0, 2, 1, 6\}$.



É o **maior** conjunto de vértices que não tem arestas entre si

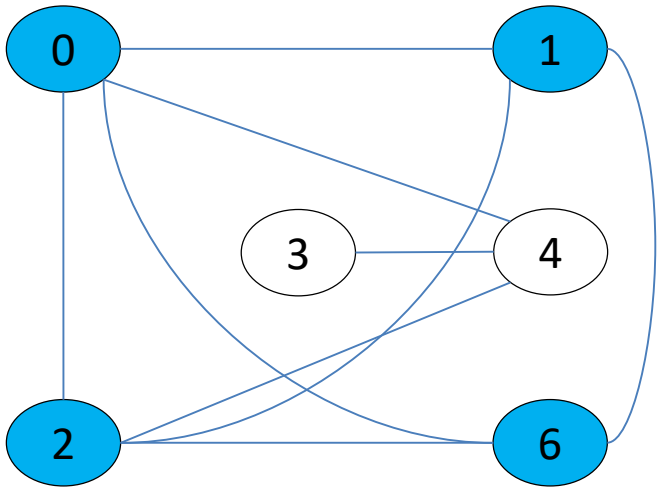
Conjuntos Independentes

- Exemplo de cardinalidade 4:
  - $S = \{0, 2, 1, 6\}$.
  - Obtendo $C(G)$



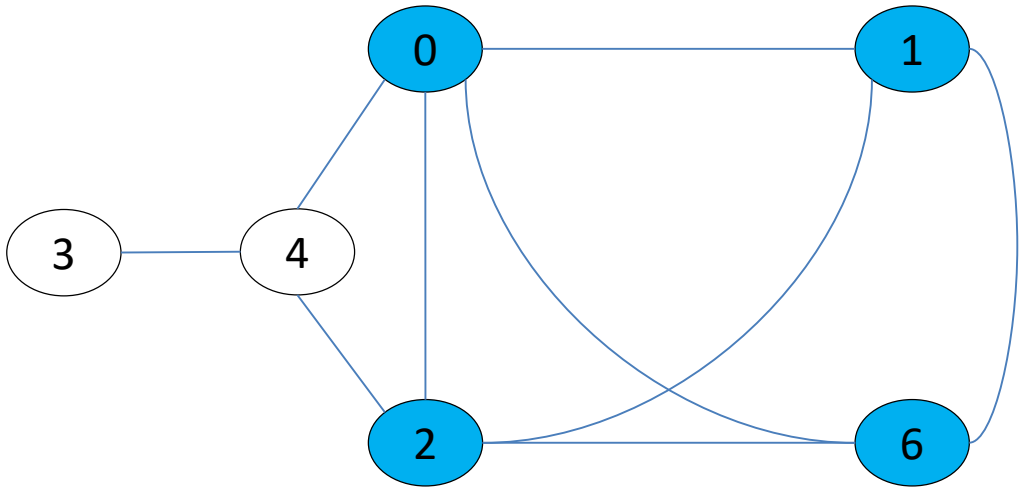
Conjuntos Independentes

- Exemplo de cardinalidade 4:
  - $S = \{0, 2, 1, 6\}$.
  - Obtendo $C(G)$



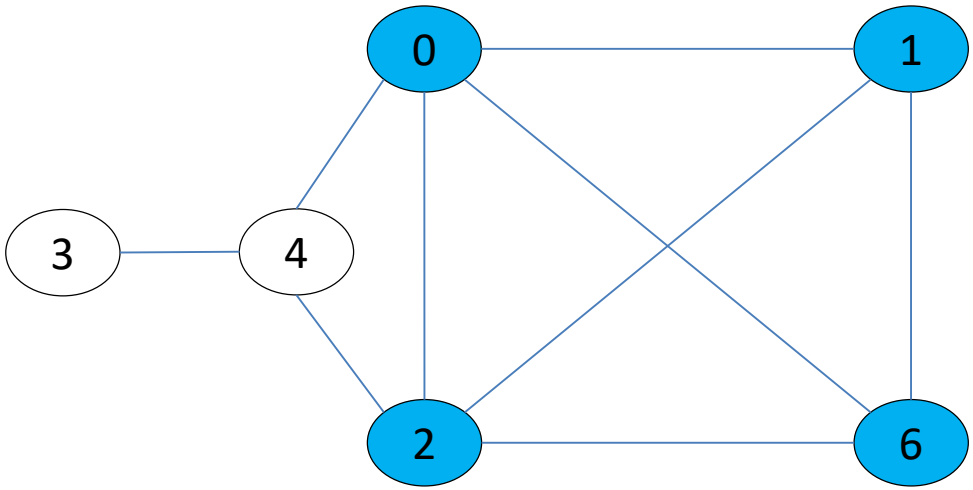
Conjuntos Independentes

- Exemplo de cardinalidade 4:
  - $S = \{0, 2, 1, 6\}$.
  - Obtendo $C(G)$



Conjuntos Independentes

- Exemplo de cardinalidade 4:
  - $S = \{0, 2, 1, 6\} = Clique\ em\ C(G)$.
  - Obtendo $C(G)$



Transformação Polinomial

- Problema de decisão: Seja $G = (V, E)$ um grafo e um inteiro k, existe um conjunto independente com no mínimo k vértices?

- Problema de decisão: Seja $G = (V, E)$ um grafo e um inteiro k, existe uma clique em G com no mínimo k vértices?

Transformação Polinomial

- Sejam P_1 o problema da Clique e P_2 o problema do conjunto independente.
- Transformação polinomial de P_1 para P_2 ($P_1 \propto P_2$)
  - Seja uma instância genérica da Clique definida por um grafo $G = (V, E)$ e um inteiro $k > 0$.
  - Criamos uma instância do conjunto independente considerando o grafo complementar $\bar{G}$ de G e o mesmo inteiro k.
- A transformação é polinomial pois:
  - O grafo complementar $\bar{G}$ pode ser obtido em tempo polinomial ($O(V^2)$).
  - G possui uma clique de tamanho $\geq k$ se e somente se $\bar{G}$ tem conjunto independente de tamanho $\geq k$.

Transformação Polinomial

- Se existe um algoritmo que resolve o Conjunto Independente em tempo polinomial, ele pode ser usado para resolver a clique também em tempo polinomial

- Diz-se que Clique $\propto$ Conjunto Independente

Problema NP-Completo e NP Difícil

- Um problema P_1 é NP-Completo se:
  1) $P_1 \in NP$
  2) $P' \propto P_1$ para todo $P' \in$ NP-Completo

- Quando P_1 é NP-Difícil?
  – Quando apenas a propriedade 2 for válida.

Problema da Parada

- Dado um algoritmo determinista qualquer A com entrada E, A termina ou entra em loop infinito?

- Problema NP-difícil mas que não é NP-Completo.
  - Problema indecidível: não há algoritmo de qualquer complexidade que o resolva

SAT $\propto$ Parada

- Problema da Satisfabilidade (SAT): dada as variáveis booleanas x1, x2, x3, x4 e a expressão S = (x1 + x2)(x2 + x3 + x4), existe alguma (pelo menos uma) atribuição de valores verdade para as variáveis de tal forma que torne a fórmula verdadeira?

SAT $\propto$ Parada

- Seja a entrada do SAT uma expressão booleana S qualquer
- Construa uma instância do problema da parada consistindo de um algoritmo A e entrada E como segue:
  - Faça E = S
  - Seja A o algoritmo:
    - Avalie todas as 2^n possibilidade de atribuição de valores booleanos para as n variáveis que compõem S
    - Se S for satisfatível (alguma das possibilidades resulta em S = true), o algoritmo termina (return)
    - Senão, ele entra em loop infinito (while(1))
- S é satisfatível se e somente se A para.

O primeiro problema NP-Completo

- CIRCUIT – SAT (problema da satisfatibilidade de circuito lógico)
  - CIRCUIT – SAT $\in$ NP
  - P $\propto$ CIRCUIT – SAT para todo P $\in$ NP
    - Prova: mostra como obter de um algoritmo polinomial A com entrada E que verifica uma candidata a solução S’ um circuito combinatorial C tal que S’ é solução se e somente se C é satisfatível
    - “simula” execução de A com C
    - Transformação em tempo polinomial

Exercício: Mostre que o Problema de Isomorfismo de Subgrafos é NP-Completo

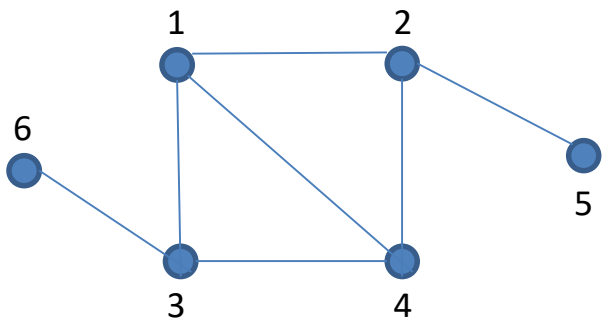
O que é um subgrafo

- Um subgrafo de um grafo G é um grafo cujo conjunto de vértices é um subconjunto dos vértices de G, e cuja relação de adjacência é um subconjunto das relações de G restrita a este subconjunto.

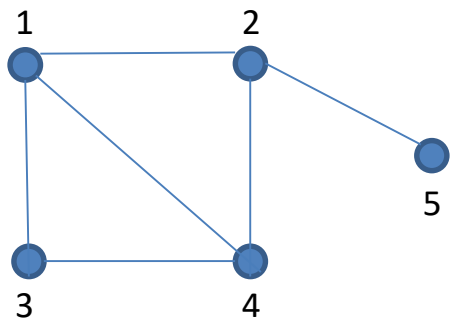
- Pode-se dizer também que um grafo H_1 contém outro grafo H_2 se algum subgrafo de H_1 é isomórfico ao grafo H_2.

O que é um subgrafo

G_1



G_2



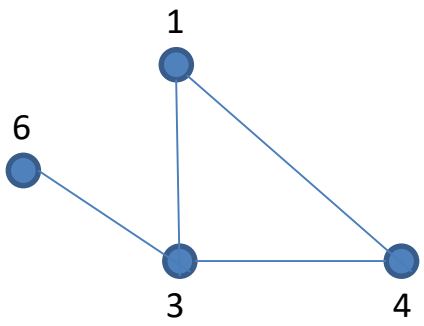
- G_2 é um subgrafo de G_1.

Isomorfismo de Grafos

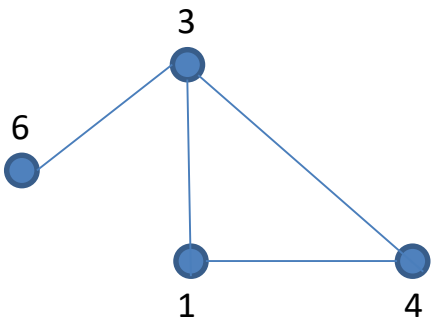
- Dois grafos G_1 e G_2 são isomórficos, se existe uma correspodência um para um entre os vértices dos grafos, talque dois vértices são adjacentes em G_1 se, e somente se, seus conrrespondentes vértices em G_2 são também adjacentes.

Isomorfismo de Grafos

G_1

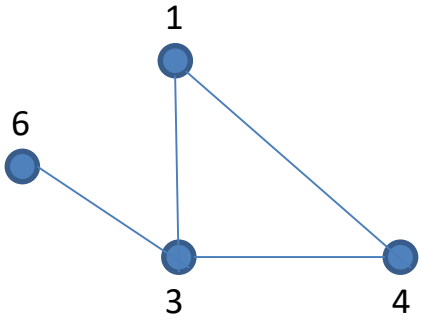


G_2

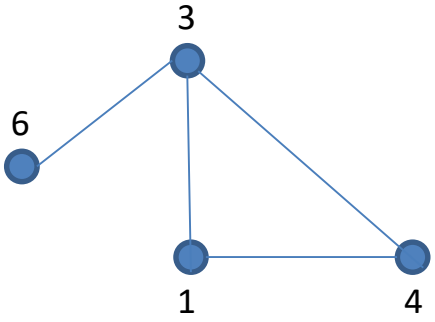


Isomorfismo de Grafos

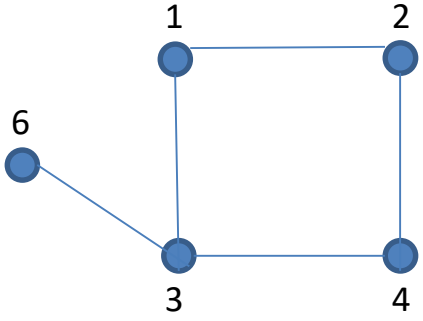
G_1



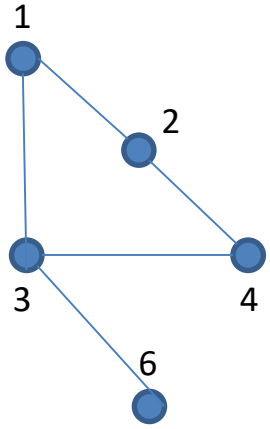
G_2



H_1



H_2



Problema de Isomorfismo de Subgrafos

- Definição (Problema de decisão):
  - Dado um grafo $H_1 = (V_1, E_1)$ e um grafo $H_2 = (V_2, E_2)$.
  - H_1contém um subgrafo isomórfico à H_2?
  - Respostas possíveis:
    - Sim, caso contenha; ou
    - Não, caso contrário.
- Objetivo:
  - Provar que o Problema de Isomorfismo de Subgrafos (IS) é um problema NP-Completo.

Prova problema NP-Completo

- A prova é divida em duas partes:
  - Provar que IS é NP;
  - Encontrar um problema X conhecido da classe de problemas NP-Completo e então reduzir X ao IS

Provar que IS é NP

```
//verifica se H1 é um grafo isomórfico a algum subgrafo de H2 (de tamanho k)
verifica_isomorfismo (H1, H2, k)
{
        //verifica se o subgrafo isomórfico tem tamanho k
        if (k != |H1|)
                return false;

        //verifica se os vértices de H1 estão em H2
        para todo v em V(H1) faça
                if (!V(H2).contem(v))
                        return false;

        //verifica se as arestas de H1 estão em H2
        para todo e(u,v) em E(H1) faça
                if (!E(H2).contem(e(u,v)))
                        return false;

        return true;
}
```

Tempo Computacional: O(V+E)

Transformação Polinomial

- Problema NP-Completo: Clique.
- Clique:
  - Uma Clique de um grafo G é um subgrafo completo de G.

- Seja uma instância genérica da Clique dada por um grafo G(V,A), é necessário obter uma instância do IS.

Transformação Polinomial: Clique $\propto$ IS

- Considere (G, k) sendo uma instância de uma Clique, em que queremos determinar se o grafo G contém uma Clique de tamanho k.

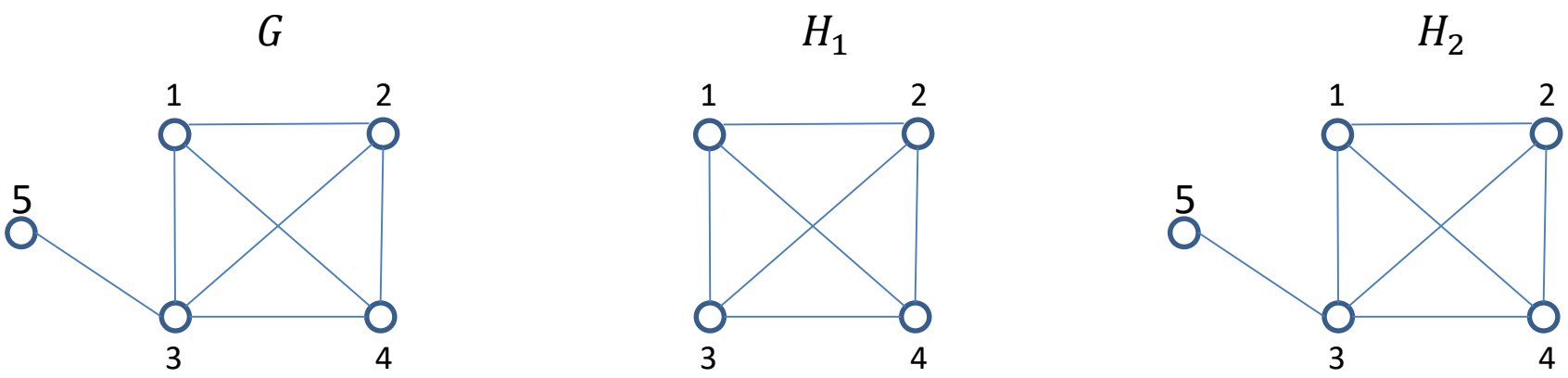
- Agora, considere H_1 e H_2 como instâncias do IS, construídas da seguinte maneira:
  - Faça H_1 um grafo completo com k vértices
  - Faça H_2 = G, sendo:
    - Todo vértice de G também é vértice de H_2.
    - Toda aresta de G também é aresta de H_2.
  - Esta operação pode ser feita em tempo polinomial.

Transformação Polinomial: Clique $\propto$ IS

- Agora será provado que:
  - Se existe solução da Clique em (G,k), existe solução do IS em (H_1, H_2);
  - Se existe solução do IS em (H_1, H_2), existe solução da Clique em (G,k).

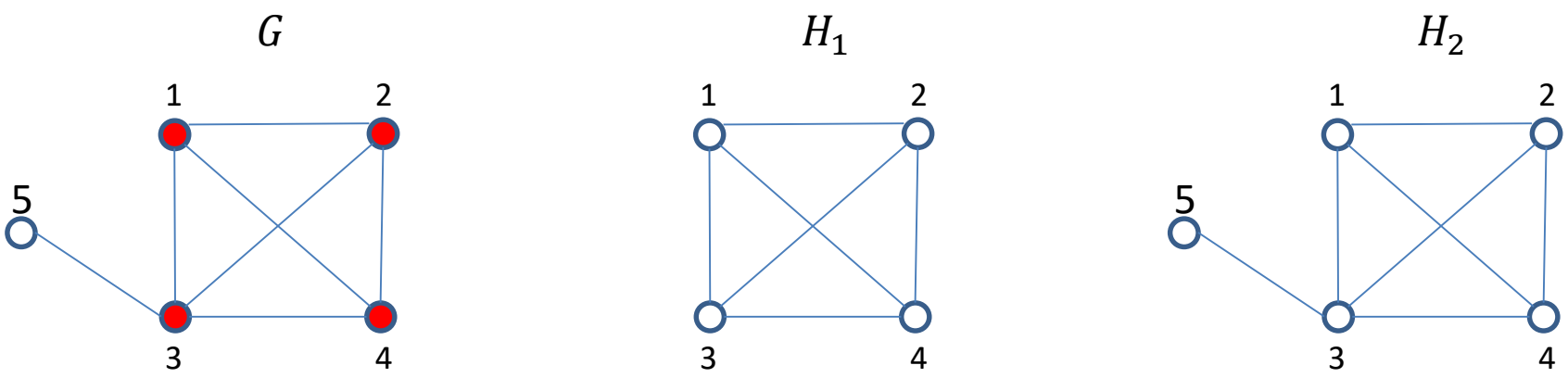
Transformação Polinomial: Clique ∝ IS

- Existe uma Clique de tamanho k em G se e somente se H_1 é um subgrafo de H_2 (pois H_2= G, e H_1 é um grafo completo de tamanho k).



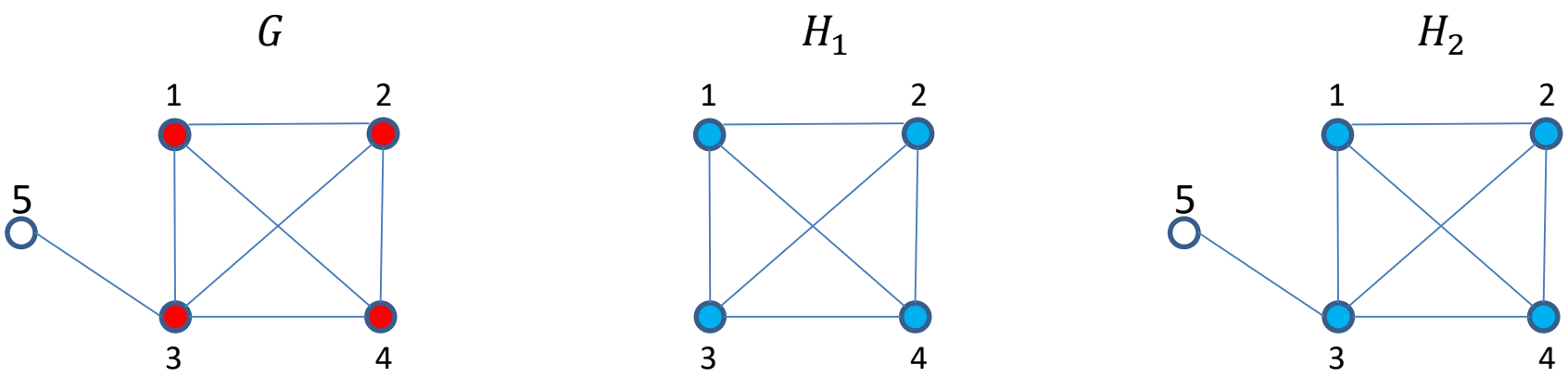
Transformação Polinomial: Clique ∝ IS

- Existe uma Clique de tamanho k em G se e somente se H_1 é um subgrafo de H_2 (pois H_2= G, e H_1 é um grafo completo de tamanho k).



Transformação Polinomial: Clique ∝ IS

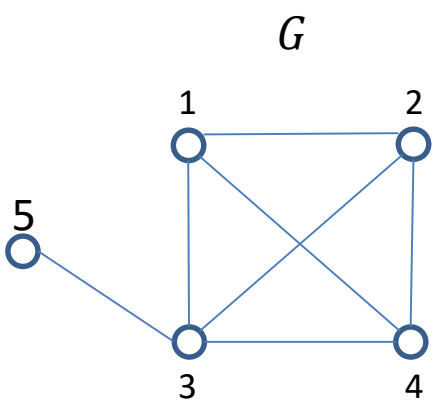
- Existe uma Clique de tamanho k em G se e somente se H_1 é um subgrafo de H_2 (pois H_2= G, e H_1 é um grafo completo de tamanho k).
- Encontrando uma Clique em G de tamanho k, resolve o problema de isomorfismo de subgrafos entre H_1 e H_2. Pois H_1 é um grafo isomórfico a um subgrafo de H_2 de tamanho k.

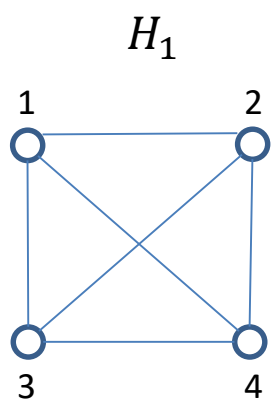


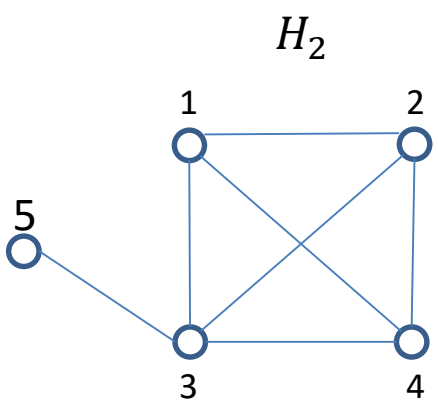
Logo, a solução da Clique em G é solução do IS entre H_1 e H_2.

Transformação Polinomial: Clique ∝ IS

- Similarmente, se existe uma solução do problema de isomorfismo de subgrafos entre H_1 e H_2 com k vértices.

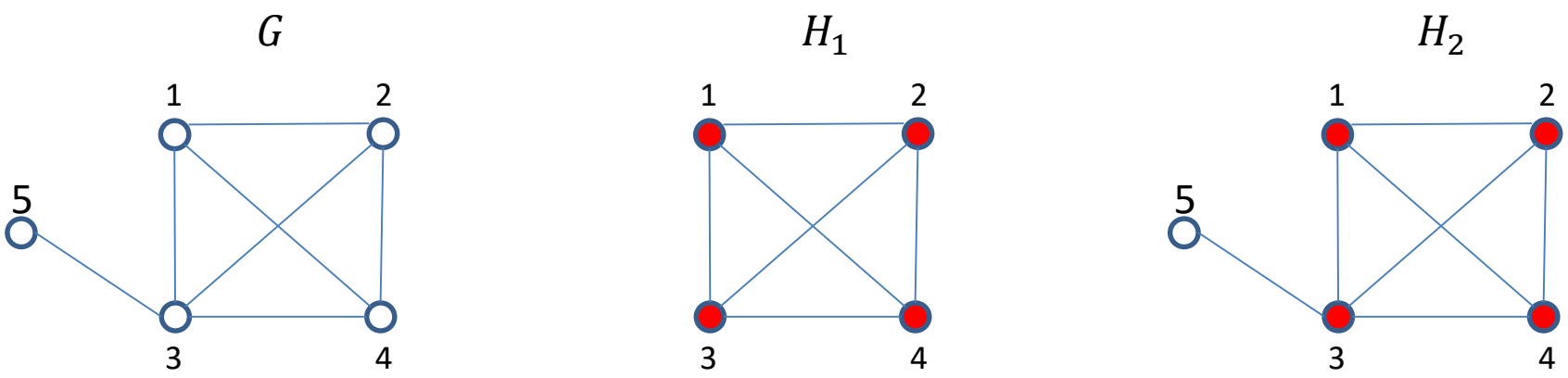






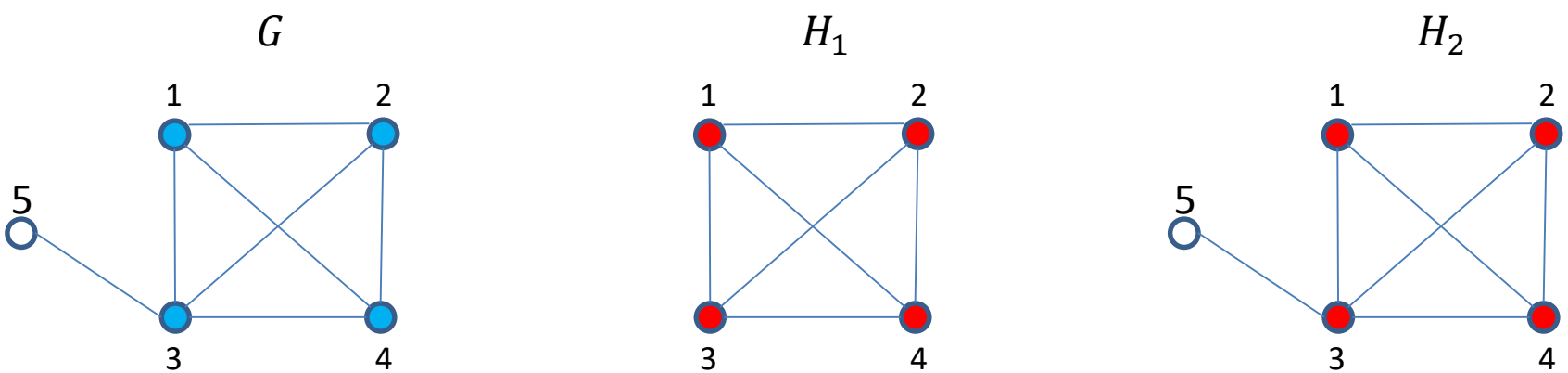
Transformação Polinomial: Clique ∝ IS

- Similarmente, se existe uma solução do problema de isomorfismo de subgrafos entre H_1 e H_2 com k vértices.



Transformação Polinomial: Clique ∝ IS

- Similarmente, se existe uma solução do problema de isomorfismo de subgrafos entre H_1 e H_2 com k vértices.
- Existe uma Clique em G com exatamente k vértices.



Por mapeamento dos vértices de H_2 em H_1, obtém-se a Clique de tamanho k em G.

Conclusão

- Portanto, o problema de Isomorfismo de Subgrafos é NP-Completo.

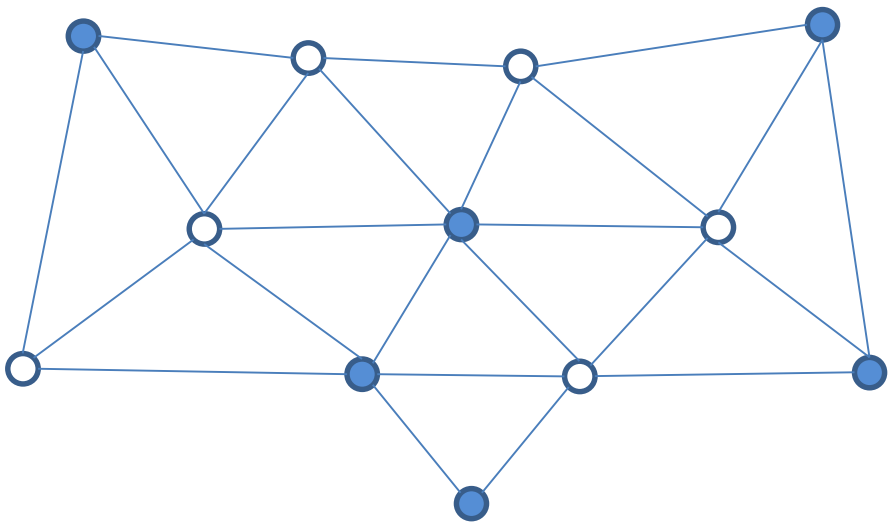
Problema da Árvore de Steiner

Árvore de Steiner

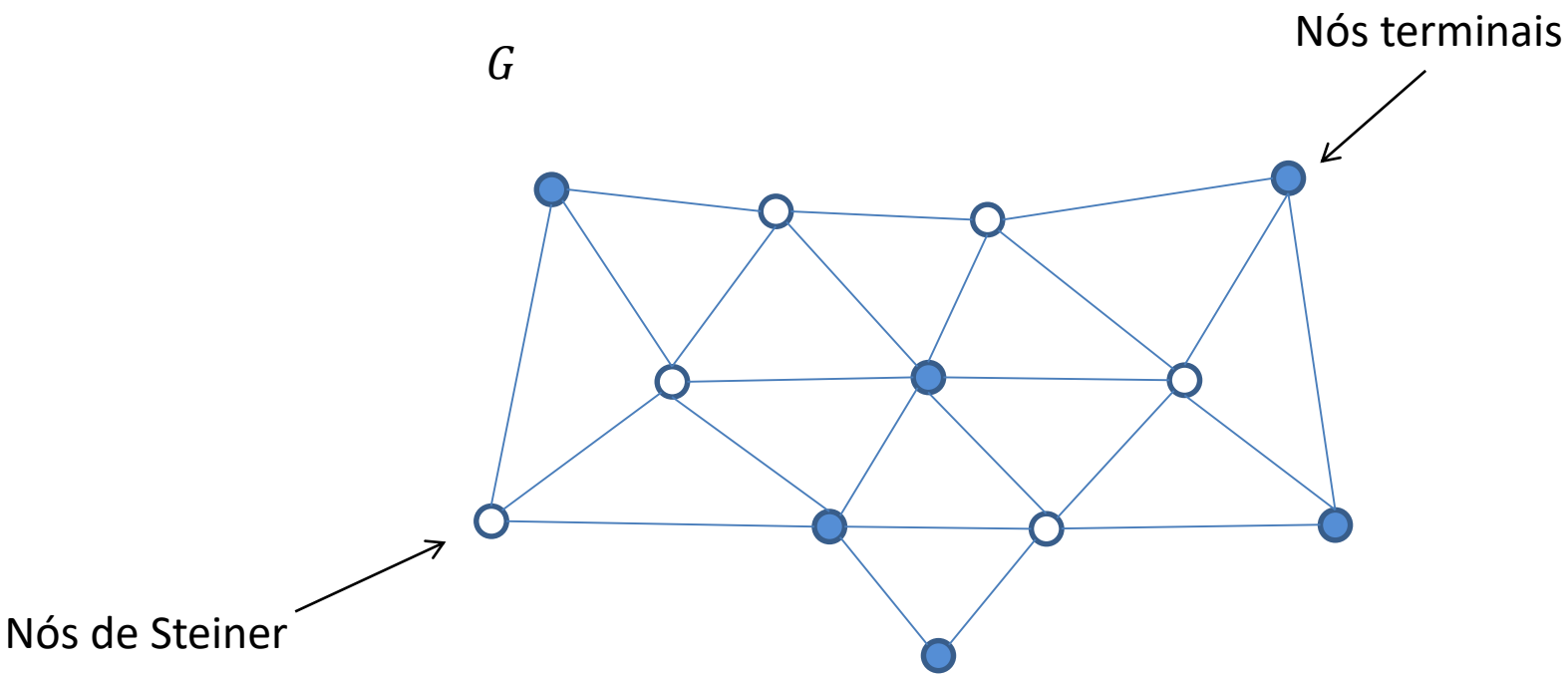
- Considere um grafo G(V, E), não dirigido.
- Considere também um subconjunto $T \subseteq V$ (de nós terminais) e o subconjunto $S \subseteq V$ (de nós de Steiner).
  - Tais que $T \cup S = V$ e $T \cap S = \emptyset$.
- A Árvore de Steiner é uma árvore em G que une todos os vértices pertencentes a T e, opcionalmente, alguns vértices de S.
- Não é necessário utilizar todos os vértices de V.
- Vértices em S são utilizados para manter a conectividade.

Árvore de Steiner

G

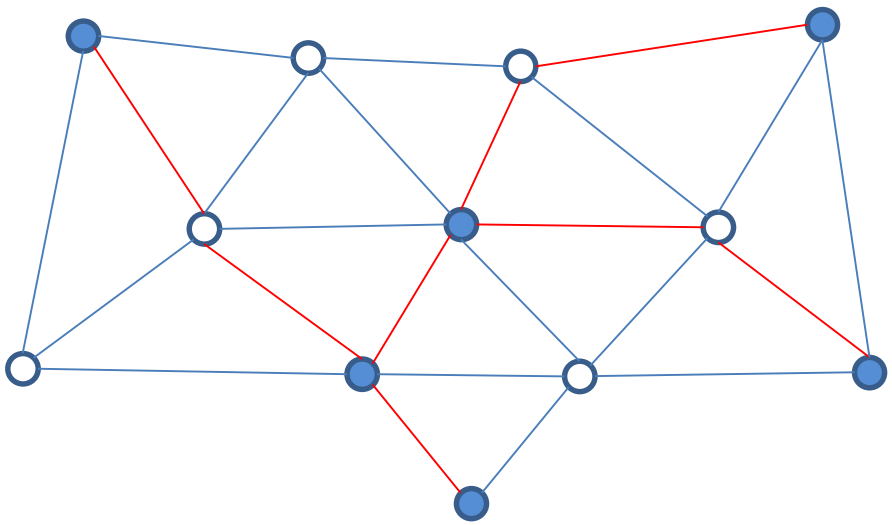


Árvore de Steiner



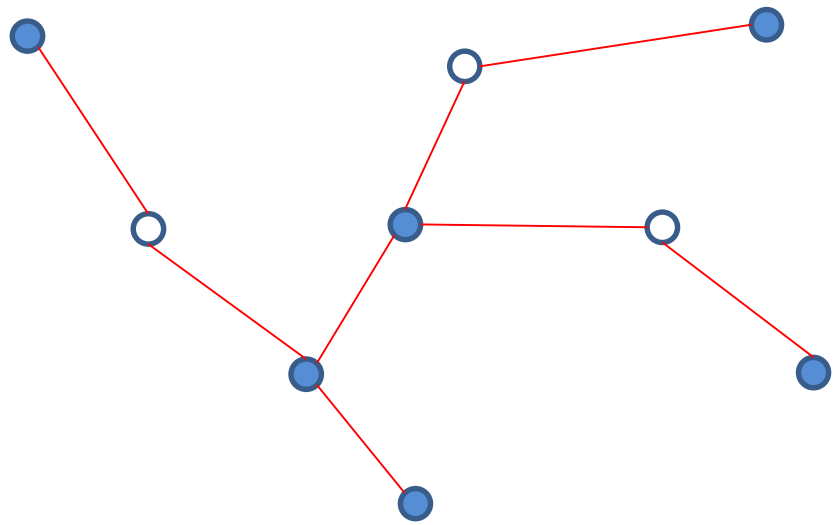
Árvore de Steiner

G



Árvore de Steiner

G



Os nós de Steiner mantém o grafo resultante conexo.

Problema da Árvore de Steiner (ST)

- Definição (Problema de decisão):
  - Considere um grafo G(V, E), um subconjunto $\mathrm{T} \subseteq V$ e um inteiro k.
  - Existe uma subárvore de G que inclui todos os vértices de T e que tenha no máximo k arestas?

Problema da Árvore de Steiner (ST)

- **Desafio**:
  - Provar que o Problema da Árvore de Steiner (ST) é um problema NP-Completo.